Week 11: Database Automation using Views and Procedures

Scenario:

The e-commerce company frequently generates reports and performs repetitive database operations such as order processing, sales calculation, and customer analysis. To improve efficiency and reduce repeated SQL coding, the company wants to automate database operations using **Views, Stored Procedures, and Functions**.

Tasks to be Done:

1. Create Views for Frequently Used Reports

1. Customer Order Report View

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_ORDER_REPORT AS
  2  SELECT
  3      C.CUSTOMER_ID,
  4      C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  5      O.ORDERID AS ORDER_ID,
  6      O.ORDERDATE AS ORDER_DATE,
  7      O.TOTALAMOUNT AS TOTAL_AMOUNT,
  8      O.ORDERSTATUS AS ORDER_STATUS
  9  FROM CUSTOMER C
 10  JOIN ORDERS O
 11      ON C.CUSTOMER_ID = O.CUSTOMERID;

View created.

SQL> SELECT *
  2  FROM CUSTOMER_ORDER_REPORT
  3  ORDER BY ORDER_DATE;
```

```
CUSTOMER_I CUSTOMER_NAME                              ORDER_I ORDER_DAT
---------- ---------------------------------------- ------- ---------
TOTAL_AMOUNT ORDER_STATUS
------------ ---------------
CUST000001 Arjun Kumar                                ORD0001 10-JAN-26
       55850 Delivered

CUST000002 Priya Sharma                               ORD0002 25-JAN-26
        3350 Delivered

CUST000003 Rahul Menon                                ORD0003 05-FEB-26
       28000 Delivered


CUSTOMER_I CUSTOMER_NAME                              ORDER_I ORDER_DAT
---------- ---------------------------------------- ------- ---------
TOTAL_AMOUNT ORDER_STATUS
------------ ---------------
CUST000004 Ananya Iyer                                ORD0004 18-FEB-26
        8300 Delivered

CUST000005 Vikram Reddy                               ORD0005 02-MAR-26
       22000 Processing

CUST000006 Meera Krishnan                             ORD0006 15-MAR-26
        7000 Delivered


CUSTOMER_I CUSTOMER_NAME                              ORDER_I ORDER_DAT
---------- ---------------------------------------- ------- ---------
TOTAL_AMOUNT ORDER_STATUS
------------ ---------------
CUST000007 Suresh Nair                                ORD0007 01-APR-26
        6500 Shipped

CUST000008 Kavya Rajan                                ORD0008 20-APR-26
        5700 Delivered

CUST000009 Karthik Subramanian                        ORD0009 05-MAY-26
       18500 Processing


CUSTOMER_I CUSTOMER_NAME                              ORDER_I ORDER_DAT
---------- ---------------------------------------- ------- ---------
TOTAL_AMOUNT ORDER_STATUS
------------ ---------------
CUST000010 Divya Pillai                               ORD0010 18-MAY-26
        5200 Delivered

CUST000011 Aditya Verma                               ORD0011 03-JUN-26
        7000 Shipped

CUST000012 Sneha Patel                                ORD0012 20-JUN-26
        2450 Delivered


CUSTOMER_I CUSTOMER_NAME                              ORDER_I ORDER_DAT
---------- ---------------------------------------- ------- ---------
TOTAL_AMOUNT ORDER_STATUS
------------ ---------------
CUST000014 Lakshmi Venkatesan                         ORD0014 15-AUG-26
       28000 Delivered

CUST000015 Nithin Murugan                             ORD0015 25-SEP-26
        8350 Processing


14 rows selected.
```

2. Product Sales Report View

```
SQL> CREATE OR REPLACE VIEW PRODUCT_SALES_REPORT AS
  2  SELECT
  3      P.PRODUCT_ID,
  4      P.PRODUCT_NAME,
  5      NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD,
  6      NVL(SUM(OD.QUANTITY * OD.PRICE), 0) AS TOTAL_REVENUE
  7  FROM PRODUCT P
  8  LEFT JOIN ORDERDETAIL OD
  9      ON P.PRODUCT_ID = OD.PRODUCTID
 10  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME;

View created.

SQL> SELECT *
  2  FROM PRODUCT_SALES_REPORT
  3  ORDER BY TOTAL_REVENUE DESC;
```

```
PRODUCT PRODUCT_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
P000005 Smartphone                                                     6
       168000

P000001 Laptop                                                         1
        55000

P000010 Tablet                                                         1
        22000


PRODUCT PRODUCT_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
P000013 Gaming Monitor                                                 1
        18500

P000008 Smart Watch                                                    2
        13000

P000015 Wireless Earbuds                                               3
        10500


PRODUCT PRODUCT_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
P000012 External Hard Disk                                             2
        10400

P000009 Headphones                                                     2
         8400

P000004 Bluetooth Speaker                                              1
         3200


PRODUCT PRODUCT_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
P000006 USB Cable                                                      6
         2700

P000003 Mechanical Keyboard                                            1
         2500

P000014 Laptop Stand                                                   1
         1500


PRODUCT PRODUCT_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
P000002 Wireless Mouse                                                 1
          850

P000007 Power Bank                                                     0
            0

P000011 Web Camera                                                     0
            0


15 rows selected.
```

3. Payment Status View

```
SQL> CREATE OR REPLACE VIEW PAYMENT_STATUS_REPORT AS
  2  SELECT
  3      O.ORDERID AS ORDER_ID,
  4      PM.PAYMENTMODE AS PAYMENT_MODE,
  5      PM.PAYMENTSTATUS AS PAYMENT_STATUS,
  6      PM.TRANSACTIONAMOUNT AS TRANSACTION_AMOUNT
  7  FROM ORDERS O
  8  LEFT JOIN PAYMENT PM
  9      ON O.ORDERID = PM.ORDERID;

View created.

SQL> SELECT *
  2  FROM PAYMENT_STATUS_REPORT
  3  ORDER BY ORDER_ID;

ORDER_I PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- --------------- ---------- ------------------
ORD0001 Credit Card     Completed               55850
ORD0002 UPI             Completed                3350
ORD0003
ORD0004 Debit Card      Completed                8300
ORD0005
ORD0006 Net Banking     Completed                7000
ORD0007
ORD0008 COD             Completed                5700
ORD0009
ORD0010 UPI             Completed                5200
ORD0011

ORDER_I PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- --------------- ---------- ------------------
ORD0012 Credit Card     Completed                2450
ORD0014 Debit Card      Completed               28000
ORD0014 Debit Card      Failed                  28000
ORD0015

15 rows selected.
```

2. Develop Stored Procedures for Order Processing

1. Add New Customer Order

```
SQL> CREATE OR REPLACE PROCEDURE ADD_NEW_ORDER
  2  (
  3      P_ORDER_ID IN VARCHAR2,
  4      P_CUSTOMER_ID IN VARCHAR2,
  5      P_PRODUCT_ID IN VARCHAR2,
  6      P_QUANTITY IN NUMBER,
  7      P_PRICE IN NUMBER
  8  )
  9  AS
 10      V_TOTAL NUMBER(9,2);
 11  BEGIN
 12      -- Calculate order amount
 13      V_TOTAL := P_QUANTITY * P_PRICE;
 14
 15      -- Insert order
 16      INSERT INTO ORDERS
 17      (
 18          ORDERID,
 19          CUSTOMERID,
 20          ORDERDATE,
 21          TOTALAMOUNT,
 22          ORDERSTATUS
 23      )
 24      VALUES
 25      (
 26          P_ORDER_ID,
 27          P_CUSTOMER_ID,
 28          SYSDATE,
 29          V_TOTAL,
 30          'Processing'
 31      );
 32
 33      -- Insert order details
 34      INSERT INTO ORDERDETAIL
 35      (
 36          ORDER_DETAIL_ID,
 37          ORDERID,
 38          PRODUCTID,
 39          QUANTITY,
 40          PRICE
 41      )
 42      VALUES
 43      (
 44          'OD' || SUBSTR(P_ORDER_ID, 4),
 45          P_ORDER_ID,
 46          P_PRODUCT_ID,
 47          P_QUANTITY,
 48          P_PRICE
 49      );
 50
 51      DBMS_OUTPUT.PUT_LINE('Order inserted successfully');
 52      DBMS_OUTPUT.PUT_LINE('Total Amount: ' || V_TOTAL);
 53
 54  EXCEPTION
 55      WHEN OTHERS THEN
 56          RAISE;
 57  END;
 58  /

Procedure created.
```

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2      ADD_NEW_ORDER(
  3          'ORD0016',
  4          'CUST000001',
  5          'P000002',
  6          2,
  7          850
  8      );
  9  END;
 10  /
Order inserted successfully
Total Amount: 1700

PL/SQL procedure successfully completed.
```

2. Update Order Status

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS
  2  (
  3      P_ORDER_ID IN VARCHAR2,
  4      P_NEW_STATUS IN VARCHAR2
  5  )
  6  AS
  7      V_OLD_STATUS VARCHAR2(15);
  8  BEGIN
  9      SELECT ORDERSTATUS
 10      INTO V_OLD_STATUS
 11      FROM ORDERS
 12      WHERE ORDERID = P_ORDER_ID
 13      FOR UPDATE;
 14
 15      IF V_OLD_STATUS = 'Processing'
 16         AND P_NEW_STATUS = 'Shipped' THEN
 17
 18          UPDATE ORDERS
 19          SET ORDERSTATUS = P_NEW_STATUS
 20          WHERE ORDERID = P_ORDER_ID;
 21
 22      ELSIF V_OLD_STATUS = 'Shipped'
 23         AND P_NEW_STATUS = 'Delivered' THEN
 24
 25          UPDATE ORDERS
 26          SET ORDERSTATUS = P_NEW_STATUS
 27          WHERE ORDERID = P_ORDER_ID;
 28
 29      ELSE
 30          RAISE_APPLICATION_ERROR(
 31              -20001,
 32              'Invalid order status transition'
 33          );
 34      END IF;
 35
 36      DBMS_OUTPUT.PUT_LINE(
 37          'Order status updated from ' ||
 38          V_OLD_STATUS || ' to ' || P_NEW_STATUS
 39      );
 40
 41  EXCEPTION
 42      WHEN NO_DATA_FOUND THEN
 43          RAISE_APPLICATION_ERROR(-20002, 'Order ID not found');
 44  END;
 45  /

Procedure created.

SQL> BEGIN
  2      UPDATE_ORDER_STATUS('ORD0016', 'Shipped');
  3  END;
  4  /
Order status updated from Processing to Shipped

PL/SQL procedure successfully completed.

SQL> BEGIN
  2      UPDATE_ORDER_STATUS('ORD0016', 'Delivered');
  3  END;
  4  /
Order status updated from Shipped to Delivered

PL/SQL procedure successfully completed.

SQL> SELECT ORDERID, ORDERSTATUS
  2  FROM ORDERS
  3  WHERE ORDERID = 'ORD0016';

ORDERID ORDERSTATUS
------- ---------------
ORD0016 Delivered
```

3. Generate Customer Purchase History

```
SQL> CREATE OR REPLACE PROCEDURE GET_CUSTOMER_ORDERS
  2  (
  3      P_CUSTOMER_ID IN VARCHAR2,
  4      P_RESULT OUT SYS_REFCURSOR
  5  )
  6  AS
  7  BEGIN
  8      OPEN P_RESULT FOR
  9          SELECT
 10              O.ORDERID,
 11              O.ORDERDATE,
 12              P.PRODUCT_NAME,
 13              OD.QUANTITY,
 14              OD.PRICE,
 15              OD.QUANTITY * OD.PRICE AS PURCHASE_AMOUNT,
 16              O.TOTALAMOUNT AS ORDER_TOTAL,
 17              O.ORDERSTATUS
 18          FROM ORDERS O
 19          JOIN ORDERDETAIL OD
 20              ON O.ORDERID = OD.ORDERID
 21          JOIN PRODUCT P
 22              ON OD.PRODUCTID = P.PRODUCT_ID
 23          WHERE O.CUSTOMERID = P_CUSTOMER_ID
 24          ORDER BY O.ORDERDATE;
 25  END;
 26  /

Procedure created.

SQL> VARIABLE RESULT REFCURSOR;
SQL>
SQL> EXEC GET_CUSTOMER_ORDERS('CUST000001', :RESULT);

PL/SQL procedure successfully completed.

SQL>
SQL> PRINT RESULT;

ORDERID ORDERDATE PRODUCT_NAME                                        QUANTITY
------- --------- ------------------------------------------------ ----------
     PRICE PURCHASE_AMOUNT ORDER_TOTAL ORDERSTATUS
---------- --------------- ----------- ---------------
ORD0001 10-JAN-26 Laptop                                                   1
     55000           55000       55850 Delivered

ORD0001 10-JAN-26 Wireless Mouse                                           1
       850             850       55850 Delivered

ORD0016 11-NOV-26 Wireless Mouse                                           2
       850            1700        1700 Delivered
```

3. Create Functions for Sales Calculations

1. Calculate Total Sales

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_TOTAL_SALES
  2  (
  3      P_START_DATE IN DATE,
  4      P_END_DATE IN DATE
  5  )
  6  RETURN NUMBER
  7  AS
  8      V_TOTAL NUMBER(12,2);
  9  BEGIN
 10      SELECT NVL(SUM(TOTALAMOUNT), 0)
 11      INTO V_TOTAL
 12      FROM ORDERS
 13      WHERE ORDERDATE >= P_START_DATE
 14        AND ORDERDATE < P_END_DATE + 1;
 15
 16      RETURN V_TOTAL;
 17  END;
 18  /

Function created.

SQL> SELECT CALCULATE_TOTAL_SALES(
  2      DATE '2026-01-01',
  3      DATE '2026-03-31'
  4  ) AS TOTAL_SALES
  5  FROM DUAL;

TOTAL_SALES
-----------
     124500
```

2. Calculate Customer Total Spending

```
SQL> CREATE OR REPLACE FUNCTION CUSTOMER_TOTAL_SPENDING
  2  (
  3      P_CUSTOMER_ID IN VARCHAR2
  4  )
  5  RETURN NUMBER
  6  AS
  7      V_TOTAL NUMBER(12,2);
  8  BEGIN
  9      SELECT NVL(SUM(TOTALAMOUNT), 0)
 10      INTO V_TOTAL
 11      FROM ORDERS
 12      WHERE CUSTOMERID = P_CUSTOMER_ID;
 13
 14      RETURN V_TOTAL;
 15  END;
 16  /

Function created.

SQL> SELECT
  2      CUSTOMER_TOTAL_SPENDING('CUST000001') AS TOTAL_SPENDING
  3  FROM DUAL;

TOTAL_SPENDING
--------------
         57550
```

3. Calculate Product Revenue

```
SQL> CREATE OR REPLACE FUNCTION PRODUCT_REVENUE
  2  (
  3      P_PRODUCT_ID IN VARCHAR2
  4  )
  5  RETURN NUMBER
  6  AS
  7      V_REVENUE NUMBER(12,2);
  8  BEGIN
  9      SELECT NVL(SUM(QUANTITY * PRICE), 0)
 10      INTO V_REVENUE
 11      FROM ORDERDETAIL
 12      WHERE PRODUCTID = P_PRODUCT_ID;
 13
 14      RETURN V_REVENUE;
 15  END;
 16  /

Function created.

SQL> SELECT
  2      PRODUCT_NAME,
  3      PRODUCT_REVENUE(PRODUCT_ID) AS TOTAL_REVENUE
  4  FROM PRODUCT
  5  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_NAME                                       TOTAL_REVENUE
-------------------------------------------------- -------------
Smartphone                                            168000
Laptop                                                 55000
Tablet                                                 22000
Gaming Monitor                                         18500
Smart Watch                                            13000
Wireless Earbuds                                       10500
External Hard Disk                                     10400
Headphones                                              8400
Bluetooth Speaker                                       3200
USB Cable                                               2700
Wireless Mouse                                          2550

PRODUCT_NAME                                       TOTAL_REVENUE
-------------------------------------------------- -------------
Mechanical Keyboard                                     2500
Laptop Stand                                            1500
Power Bank                                                 0
Web Camera                                                 0

15 rows selected.
```

4. Automate Repetitive Database Operations

```
SQL> CREATE OR REPLACE PROCEDURE PROCESS_ORDER_AUTOMATED
  2  (
  3      P_ORDER_ID IN VARCHAR2,
  4      P_DETAIL_ID IN VARCHAR2,
  5      P_CUSTOMER_ID IN VARCHAR2,
  6      P_PRODUCT_ID IN VARCHAR2,
  7      P_QUANTITY IN NUMBER,
  8      P_PRICE IN NUMBER,
  9      P_PAYMENT_ID IN VARCHAR2,
 10      P_PAYMENT_MODE IN VARCHAR2
 11  )
 12  AS
 13      V_STOCK NUMBER;
 14      V_TOTAL NUMBER(12,2);
 15  BEGIN
 16      SAVEPOINT ORDER_START;
 17
 18      -- Check and lock inventory
 19      SELECT STOCK_QUANTITY
 20      INTO V_STOCK
 21      FROM INVENTORY
 22      WHERE PRODUCT_ID = P_PRODUCT_ID
 23      FOR UPDATE;
 24
 25      -- Validate quantity
 26      IF P_QUANTITY <= 0 THEN
 27          RAISE_APPLICATION_ERROR(-20003, 'Invalid quantity');
 28      END IF;
 29
 30      -- Check stock availability
 31      IF V_STOCK < P_QUANTITY THEN
 32          RAISE_APPLICATION_ERROR(-20004, 'Insufficient stock');
 33      END IF;
 34
 35      V_TOTAL := P_QUANTITY * P_PRICE;
 36
 37      -- Insert order
 38      INSERT INTO ORDERS
 39      VALUES (
 40          P_ORDER_ID,
 41          P_CUSTOMER_ID,
 42          SYSDATE,
 43          V_TOTAL,
 44          'Processing'
 45      );
 46
 47      -- Insert order detail
 48      INSERT INTO ORDERDETAIL
 49      VALUES (
 50          P_DETAIL_ID,
 51          P_ORDER_ID,
 52          P_PRODUCT_ID,
 53          P_QUANTITY,
 54          P_PRICE
 55      );
 56
 57      -- Reduce inventory stock
 58      UPDATE INVENTORY
 59      SET STOCK_QUANTITY = STOCK_QUANTITY - P_QUANTITY,
 60          STOCK_STATUS =
 61              CASE
 62                  WHEN STOCK_QUANTITY - P_QUANTITY <= 5
 63                      THEN 'Low Stock'
 64                  ELSE 'Available'
 65              END,
 66          LAST_UPDATED = SYSDATE
 67      WHERE PRODUCT_ID = P_PRODUCT_ID;
 68
 69      -- Synchronize product stock
 70      UPDATE PRODUCT
 71      SET STOCK_QUANTITY = STOCK_QUANTITY - P_QUANTITY
 72      WHERE PRODUCT_ID = P_PRODUCT_ID;
 73
 74      -- Insert payment record
 75      INSERT INTO PAYMENT
 76      VALUES (
 77          P_PAYMENT_ID,
 78          P_ORDER_ID,
 79          SYSDATE,
 80          P_PAYMENT_MODE,
 81          'Pending',
 82          V_TOTAL
 83      );
 84
 85      DBMS_OUTPUT.PUT_LINE('Order processed successfully');
 86      DBMS_OUTPUT.PUT_LINE('Total Amount: ' || V_TOTAL);
 87
 88  EXCEPTION
 89      WHEN NO_DATA_FOUND THEN
 90          ROLLBACK TO ORDER_START;
 91          RAISE_APPLICATION_ERROR(-20005, 'Product inventory not found');
 92
 93      WHEN OTHERS THEN
 94          ROLLBACK TO ORDER_START;
 95          RAISE;
 96  END;
 97  /

Procedure created.
```

```
QL> BEGIN
  2      PROCESS_ORDER_AUTOMATED(
  3          'ORD0022',
  4          'OD00021',
  5          'CUST000001',
  6          'P000002',
  7          2,
  8          8500,
  9          'PAY0101',
 10          'UPI'
 11      );
 12  END;
 13  /
rder processed successfully
otal Amount: 17000

L/SQL procedure successfully completed.
```